

CCA – COMPETENCE CENTRE
HTL Anichstraße

Schnuppertage

Wirtschaftsingenieure
Betriebsinformatik

Software entwickeln.
Technik beherrschen.
Wirtschaft durchschauen.

weitere Infos

V20260614 / EckhA



CCA – COMPETENCE CENTRE
HTL Anichstraße

WI-smartLight

Programmierung

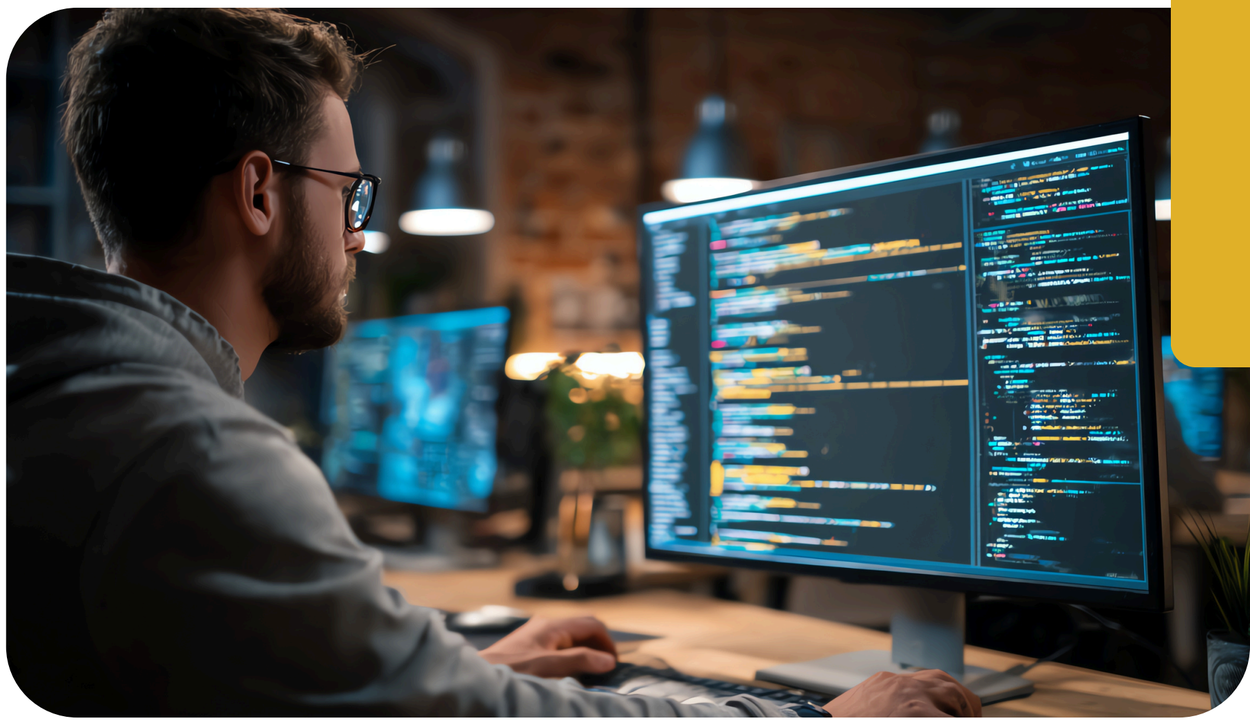
Bitte beachte

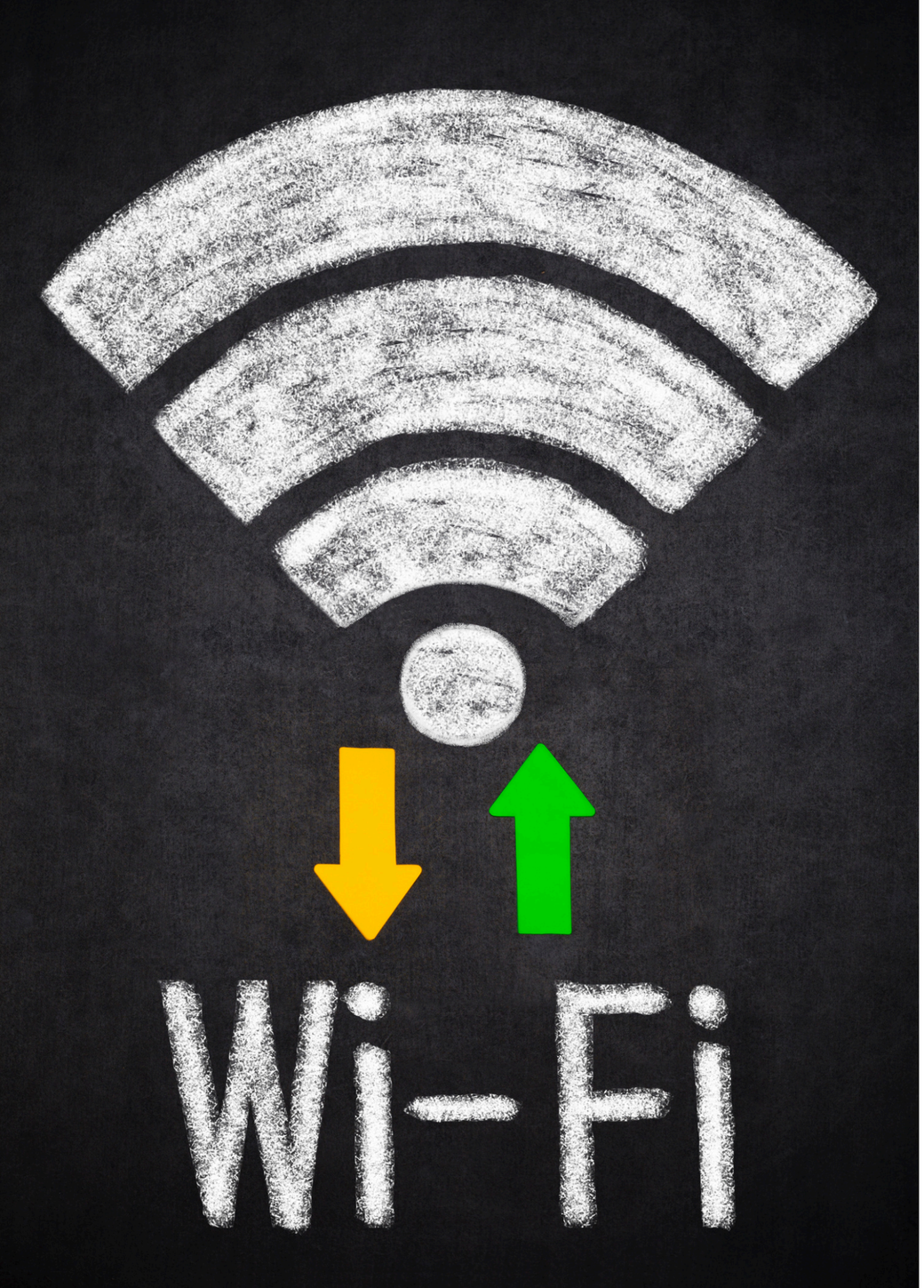
Heute gibt’s nur einen kurzen **Einblick** in die Welt des Programmierens – für **große Projekte** reicht die Zeit natürlich nicht.

Aber **im Unterricht** zeigen wir dir **Schritt für Schritt**, wie man **richtig programmiert** und eigene **coole Ideen umsetzt**.

#NichtAllesSchaffbar **#ReadyToCode**

#FromZeroToHero **#Schnupperprojekt**



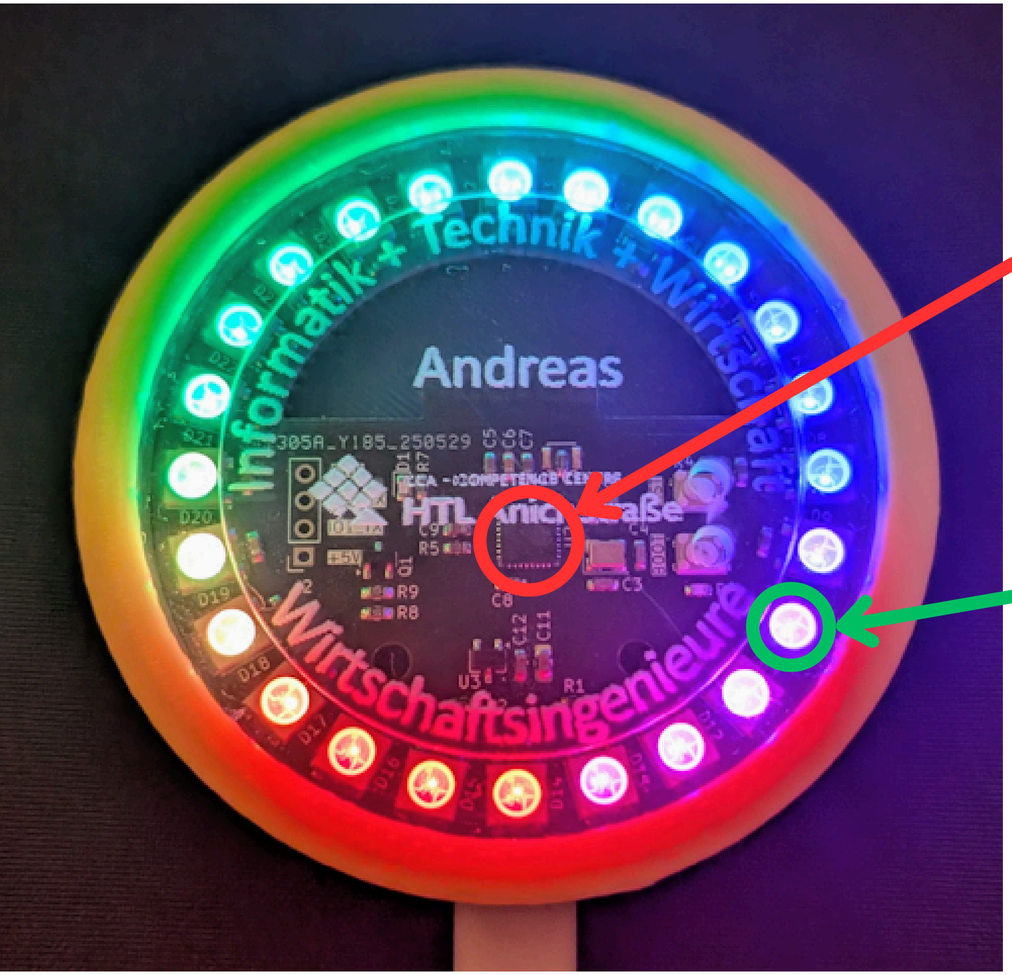


Internetzugang

Verbinde das Notebook mit dem Internet, falls dies nicht bereits erledigt ist:

WLAN:	HTL-Gast
Username:	Schnuppertag
Password:	Ausbildung@CCA

Inbetriebnahme



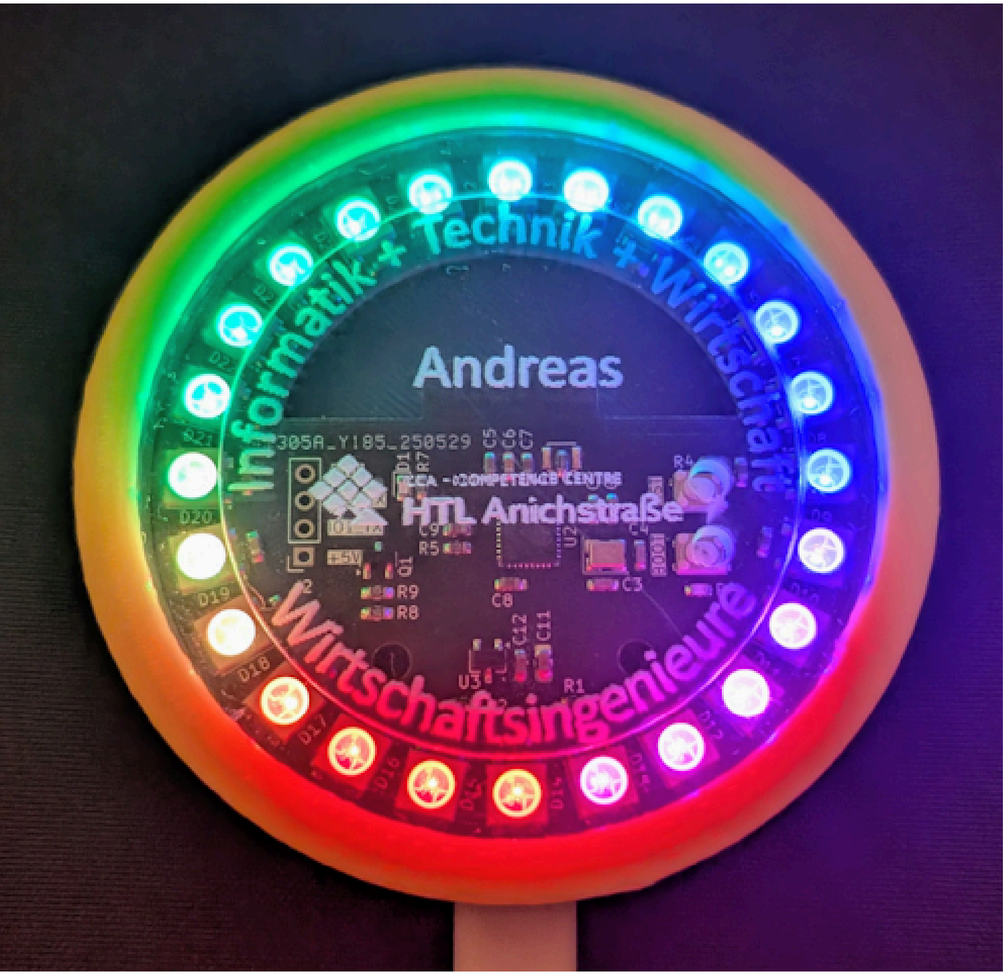
- Gespeichert wird dein Programm auf dem kleinen schwarzen Chip in der Mitte der Platine.
- Dabei handelt es sich um einen sogenannten ESP32 **Mikrocontroller**-Chip (inkl. WLAN und BLE)
- Dein **WI-smartLight Ring** besteht aus 24 einzeln ansteuerbaren RGB-LEDs.
- Jede einzelne LED kann **16.7 Millionen verschiedene Farben** in unterschiedlicher Helligkeit anzeigen.

CHALLENGE

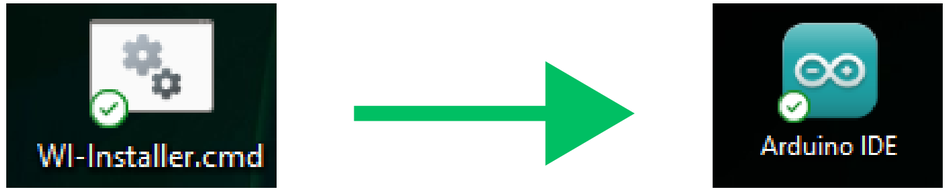
Aufgabe:

Verbinde dein WI-smartLight mithilfe des USB-C Kabels mit dem Laptop.

Neues Projekt erstellen

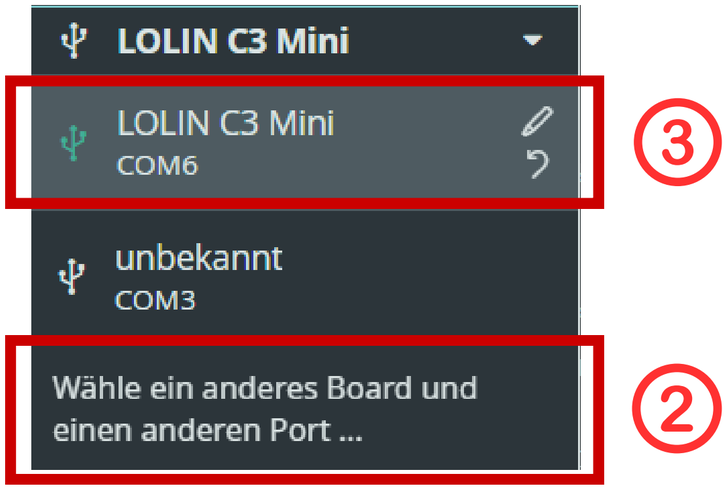


- Um den Code einfach schreiben und ausführen zu können, benötigst du eine **Programmierumgebung**.
- Wir verwenden den kostenlosen Quellcode-Editor **Arduino IDE**.
- Um ein **neues Projekt** zu erstellen klicke auf dem **Desktop** doppelt auf die Datei: **WI-Installer.cmd** und bestätige den Warnhinweis mit “**Ausführen**”.
- Anschließend wird automatisch die **Arduino IDE** mit deinem neuen Projekt gestartet.



Entwicklungsumgebung - Arduino IDE

- Wähle **LOLIN C3 Mini** und den entsprechenden **COM Port** aus (z.B. COM3).
- Sollte dein Laptop mehrere COM-Ports besitzen, wähle den Port mit der höheren Nummer (z.B. COM6)





Programmaufbau

- Das Programm besteht aus zwei Hauptteilen:
- **setup():** Wird einmal beim Start des ESP32 ausgeführt. Hier stellen wir einmalig alles Notwendige ein (z.B. LED-Ring starten, Standard-Helligkeit, usw.).
- **loop():** Wird sehr schnell ununterbrochen in einer Schleife ausgeführt! Hier kommt dann der Code für unsere Lichteffekte hin.

```
16  // Setup-Funktion - wird einmal beim Start ausgeführt
17  void setup() {
18      // hier kommt der Setup-Code hin
19
20  }
21
22  // Loop-Funktion - wird ununterbrochen ausgeführt
23  void loop() {
24      // hier kommt der Effekt-Code hin
25
26  }
```

Hinweis: Bei Code, der mit **//** gekennzeichnet ist, handelt es sich um Kommentare.

Diese werden nicht ausgeführt, sondern sind lediglich Hinweise und Erklärungen.

Vorbereitung - Setup

CHALLENGE

Aufgabe:

Füge untenstehenden Code wie abgebildet in die setup() Funktion ein.

```
/*****************************************
 * ✅ AB HIER KANNST DU BEARBEITEN! ✅
 *****************************************/

// Setup-Funktion - wird einmal beim Start a
void setup() {
    // hier kommt der Setup-Code hin
    ring.begin();
    ring.show();
    ring.setBrightness(BRIGHTNESS);
}
```

Zunächst müssen wir **einmalig** noch den LED-Ring **konfigurieren**.

Dies geschieht in der Funktion **setup()**

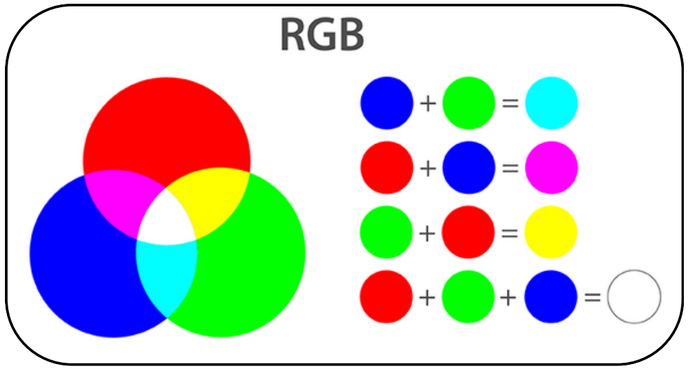
- Um die Verbindung mit dem LED-Ring zu starten:
  `ring.begin();`

- **WICHTIG:** Um die gesetzten Farben anzuzeigen muss immer dieser Befehl ausgeführt werden:
  `ring.show();`

- Standard-Helligkeit einstellen.
  WICHTIG: 0 = AUS → 255 = max.

  `ring.setBrightness(BRIGHTNESS);`

Die wichtigsten Befehle



- **Ab jetzt wird nur noch** in der Funktion ***loop()*** programmiert!

- Jede darstellbare Farbe setzt sich aus den Farben **ROT**, **GRÜN** und **BLAU** zusammen.

```
int farbe = ring.Color(ROT, GRÜN, BLAU);

int rot = ring.Color(255, 0, 0);
int gruen = ring.Color(0, 255, 0);
int blau = ring.Color(0, 0, 255);
```

- Setzt die Farbe für eine **einzelne LED**. Pos ist die Nummer der LED (0 bis 23).

```
ring.setPixelColor(Pos, Farbe)
```

- Anzeigen der Änderung:

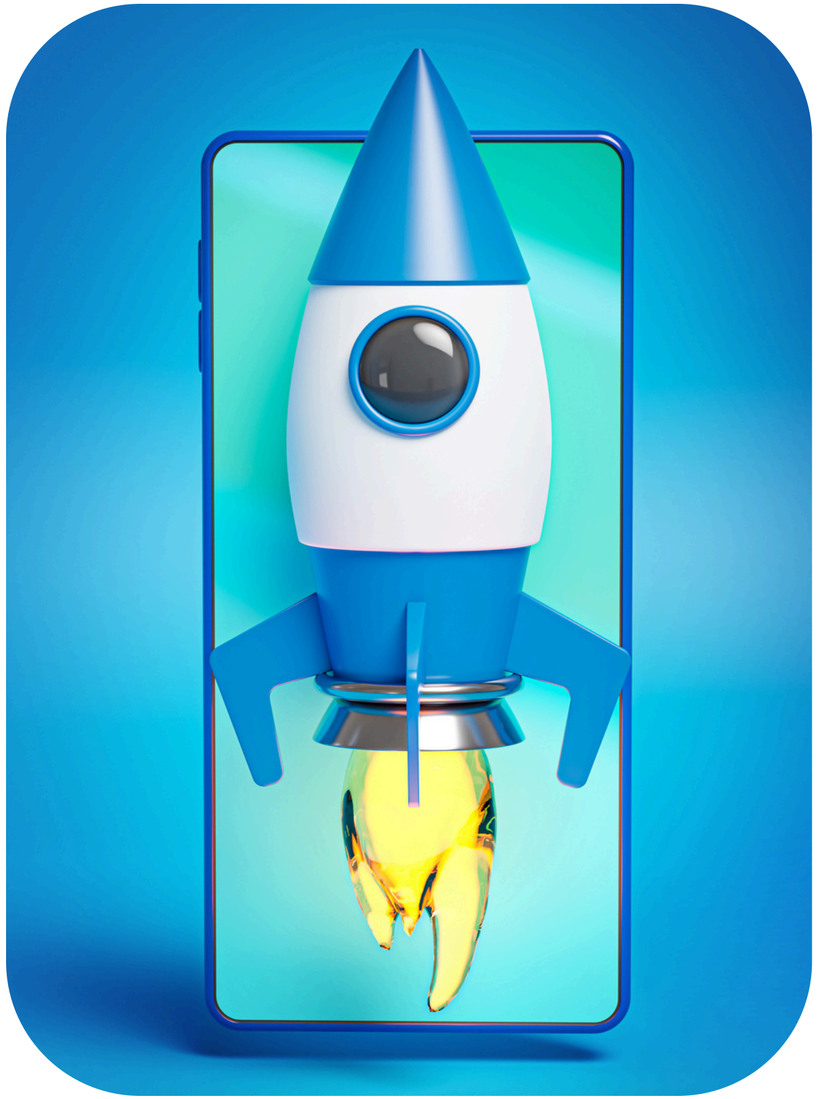
```
ring.show();
```

- Um alle LEDs auf AUS zu setzen verwende

```
ring.clear();
```

Es geht los! Testen, ob alles funktioniert

CHALLENGE



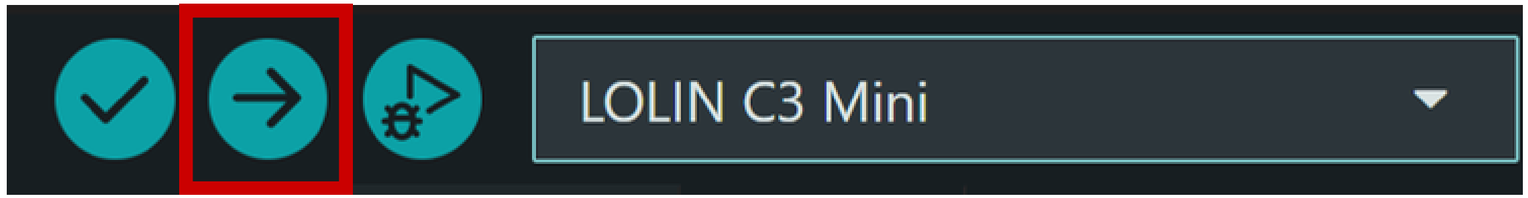
Aufgabe 1:

1. Die erste LED (= Pos 0) soll in der Farbe **rot** aufleuchten. Definiere zuerst die Farbe rot und setze dann die LED:

```
int rot = ring.Color(255, 0, 0);
ring.setPixelColor(0, rot);
```

3. Vergiss nicht die Änderungen der LEDs auch anzuzeigen! → **ring.show();**

- **Nach jeder Änderung** musst der Code **neu** auf den Controller **übertragen** werden → dauert ein wenig!



Jetzt wird’s bunt!

CHALLENGE



Aufgabe 2:

1. Setze je eine LED in den Farben rot, grün, blau und eine **weitere Farbe**, die dir gut gefällt (z.B. magenta)
2. Lasse jedoch zwischen den 4 farbigen LEDs jeweils 1 LED dunkel.
3. Für deine Lieblingsfarbe (z.B. magenta, suche im Internet nach ***“rgb color picker”*** und finde einen passenden RGB-Code.
4. Vergiss nicht die Änderungen der LEDs auch anzuzeigen! → ***ring.show();***
5. Definiere weitere Farben deiner Wahl und zeige sie an.

Warum nicht mal warten?

CHALLENGE



- Damit echte Lichteffekte entstehen, müssen zeitversetzt LEDs verändert werden.
- Um eine gewisse Zeit zu warten, verwende den Befehl:

```
delay(wartezeit);
```

Die Wartezeit wird in Millisekunden angegeben.

Aufgabe 3:

1. Passe deinen Code so an, dass die 4 LEDs für **2 Sekunden angezeigt** werden und **anschließend** für eine **halbe Sekunde AUS** sind.
2. **Erweitere** das Beispiel, dass die **1. und 3. LED** und die **2 und 4. LED** jeweils **gemeinsam abwechselnd blinken**. 1+3 EIN und 2+4 AUS … 1+3 AUS und 2+4 EIN

Alle LEDs verwenden

CHALLENGE



- Um die Farbe **aller LEDs** zu verändern, verwende den Befehl:

```
ring.fill(Farbe);
```

oder auch

```
ring.fill(ring.Color(R, G, B));
```

Aufgabe 4:

1. Lasse **alle LEDs** im folgenden Rhythmus **in einer Farbe deiner Wahl** blinken
   → 0,75s EIN - 0,5s AUS - 0,75s EIN - usw.
2. Lasse den gesamten Ring in unterschiedlichen Farben nacheinander blinken. z.B.
   → 0,5s rot - 0,75s magenta - 1s grün

Die Torte wird geteilt

CHALLENGE



- Um die Farbe **mehrerer hintereinander liegender** LEDs zu verändern, verwende den Befehl:

```
ring.fill(Farbe, Start, Anzahl);
```

oder auch

```
ring.fill(ring.Color(R,G,B), Start, Anzahl);
```

Aufgabe 5:

1. Teile den LED Ring (insgesamt 24 LEDs) in **4 gleich große Teile** und beleuchte **jeden Teil** in einer **anderen Farbe**.

Die Torte wird geteilt

CHALLENGE



- Um die Farbe **mehrerer hintereinander liegender** LEDs zu verändern, verwende den Befehl:

```
ring.fill(Farbe, Start, Anzahl);
```

oder auch

```
ring.fill(ring.Color(R,G,B), Start, Anzahl);
```

Aufgabe 6:

1. Blende **nacheinander** ein Tortenstück **nach dem anderen** ein (z.B. rot, dann rot-grün, rot-grün-blau, …). Lösche anschließen alle LEDs und beginne von vorne.
3. Blende **andere coole Muster** deiner Wahl ein (z.B. Tortenstücke rückwärts wieder ausblenden).

Schleifen

- **Schleifen** ermöglichen es, bestimmte **Teile deines Codes mehrfach auszuführen**.
- Dies ist beispielsweise hilfreich, wenn wir alle LEDs der Reihe nach einschalten möchten.
- Wir verwenden dazu eine sogenannte for-Schleife.

Diese besteht aus 3 Teilen:

- **1. Start:** Wo soll das Zählen beginnen?
  - Wir starten bei der ersten LED = Nummer 0: **int led = 0;**
- **2. Bedingung:** Wie oft soll der Code wiederholt werden?
  - Bis die letzte LED erreicht ist → solange **led < 24**
- **3. Schritt:** Was soll nach jeder Wiederholung passieren?
  - Eine LED weiterspringen (**led++**) → erhöht den Wert von led jedes mal um 1

```
for (int led = 0; led < 24; led++ ) {
  hier kommt dein Code, der wiederholt werden soll
}
```

Farbwischen

CHALLENGE



- Hilfe:

```
for (int led = 0; led < 24; led++ ) {
  hier kommt dein Code, der wiederholt wird
}
```

Aufgabe 7:

1. Erstelle mit Hilfe der for-Schleife einen Farbwisch Effekt, bei dem **nacheinander alle LEDs** in einer Farbe deiner Wahl **der Reihe nach eingeschalten** werden, bis der Ring voll ist. Warte zwischen den LEDs jeweils 100ms.
2. Schalte anschließend alle LEDs für 0,5s aus.

Farbwischen – hin und zurück

CHALLENGE

- Hilfe:

```
for (int led = 0; led < 24; led++ ) {
  hier kommt dein Code, der wiederholt wird
}
```



Aufgabe 8: (Erweitere das Beispiel 7 von vorhin.)

1. **Definiere** zuerst eine **Farbe** mit dem Namen **“aus”**, welche die LED ausschaltet → welcher RGB-Code?
2. Wenn der **Ring voll** ist, **schalte die LEDs in umgekehrter Reihenfolge** der Reihen nach wieder **aus**.
   Was denkst du, wie könnte die for-Schleife dafür aussehen?
3. **Optional:** Setze die LEDs auf dem Weg zurück nicht auf aus, sondern auf eine anderer Farbe.

Dimmen

CHALLENGE

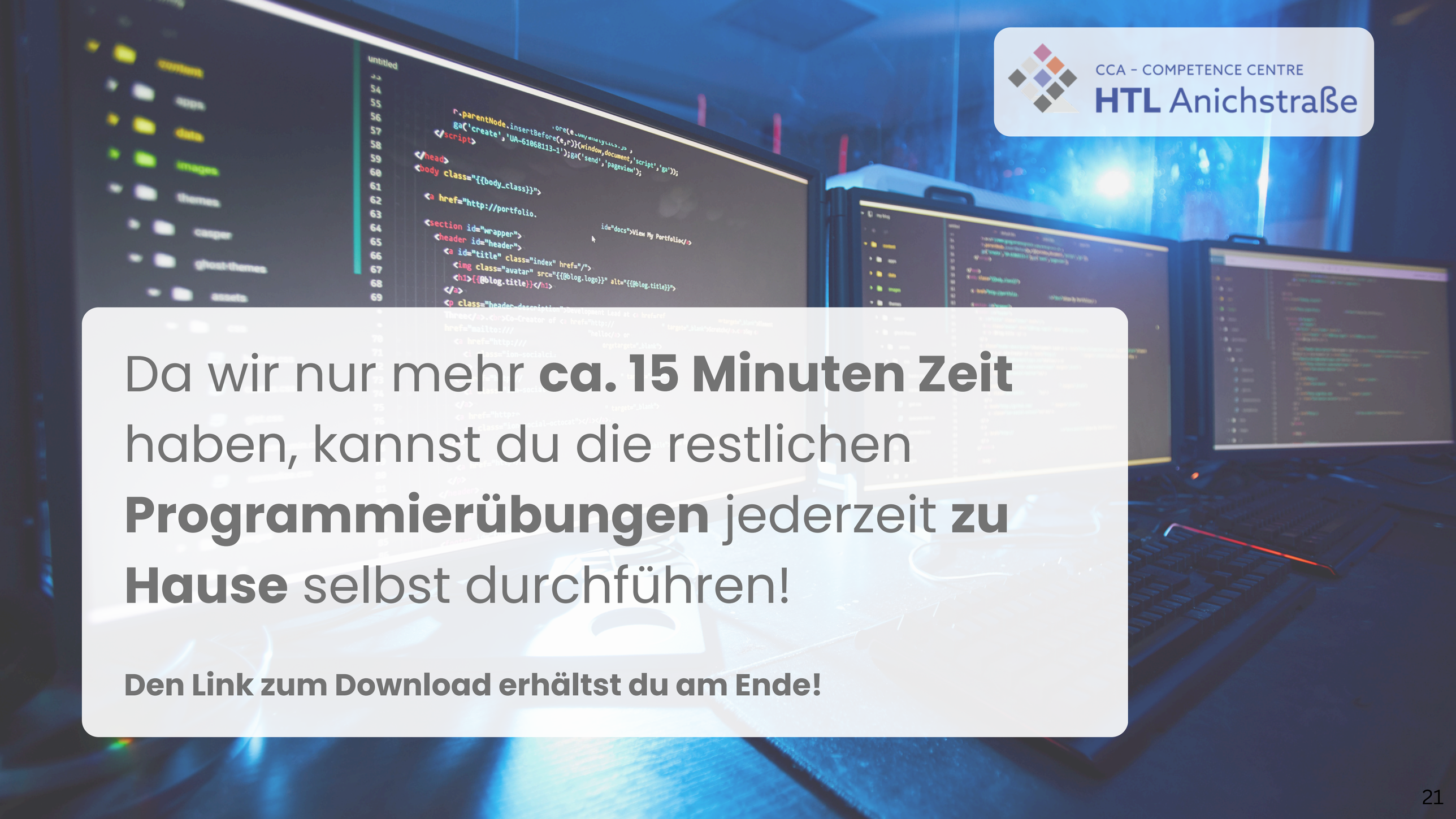


- Mit Hilfe von Schleifen können die LEDs auch ganz einfach gedimmt werden.

```
for (int hell = 1; hell < 100; hell++ ) {
  hier kommt dein Code, der wiederholt wird
}
```

Aufgabe 9:

1. Unterteile den LED-Ring in 3 beliebig große Teile mit unterschiedlichen Farben.
2. Dimme anschließend die LEDs von ganz dunkel (1) nach ganz hell (100) und wieder zurück.
3. Schalte anschließend alle LEDs für 0,5s aus.



CCA – COMPETENCE CENTRE
HTL Anichstraße

Da wir nur mehr **ca. 15 Minuten Zeit** haben, kannst du die restlichen **Programmierübungen** jederzeit **zu Hause** selbst durchführen!

Den Link zum Download erhältst du am Ende!

Alles geht auf Knopfdruck



- Dein WI-smartLight verfügt über einen **Taster**, den wir auslesen können → der untere der beiden Taster.
- Um den Taster zu aktivieren, füge folgende Zeile in der Funktion **setup()** hinzu:

```
pinMode(BUTTON_PIN, INPUT_PULLUP);
```

- Damit ist der Taster einsatzbereit.
- Um den Status (gedrückt oder nicht gedrückt) des Tasters abzufragen verwende:

```
int taster = digitalRead(BUTTON_PIN);
```

- in der Variablen **taster** steht nun der Zustand des Buttons (GEDRÜCKT oder NICHT GEDRÜCKT)

Entscheidungen treffen

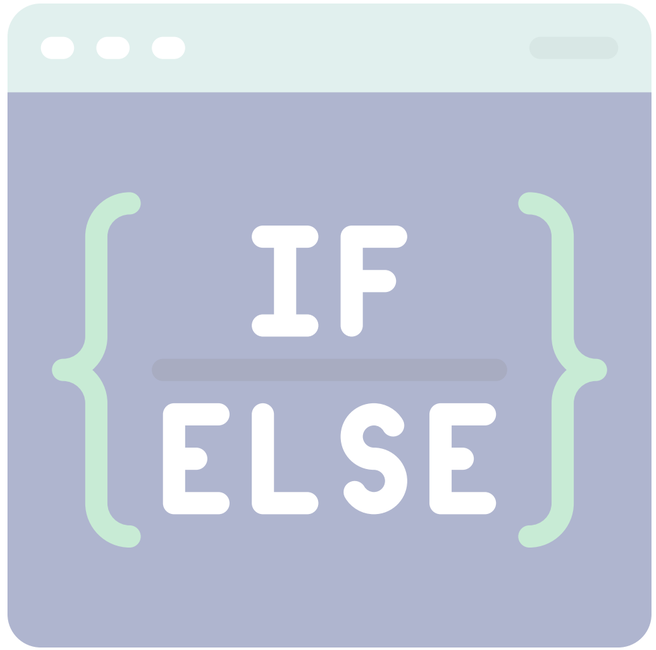
- Wie auch im realen Leben, müssen beim Programmieren immer wieder Entscheidungen getroffen werden.
- Dazu werden sogenannte **if/else-Anweisungen** verwendet.
- Wenn (if) eine Bedingung wahr ist führe diesen Code aus, ansonsten (else) führe einen anderen Code aus.

```
int taster = digitalRead(BUTTON_PIN);

if (taster == LOW) {    // Wenn Taster gedrückt
  Code für LEDs ein     // z.B. alle LEDs einschalten
  delay(200)            // Taster entprellen

} else {                // ansonsten
  Code für LEDs aus     // z.b. alle LEDs ausschalten

}
```



Entscheidungen treffen

CHALLENGE



- Hilfe:

```
int taster = digitalRead(BUTTON_PIN);
if (taster == LOW) {
  ...
} else {
  ...
}
```

Aufgabe 10:

1. **Wenn** der Button **gedrückt** ist, sollen **alle LEDs** in einer beliebigen Farbe deiner Wahl **EIN** sein. Wenn der Button **nicht gedrückt** ist sind **alle LEDs aus**.
2. Wenn gedrückt → 5x blinken (z.B. 1s EIN → 0,5s AUS).

Dynamischen Geschwindigkeit

CHALLENGE



- Nun möchten wir die Geschwindigkeit mit dem Taster steuern.
- Dazu definieren wir eine globale Variable **wartezeit** (oberhalb von setup) und setzen diese auf den Wert 200

```
int wartezeit = 200;
```

Aufgabe 11:

1. Jedes mal, **wenn der Taster gedrückt** wird, soll die **Wartezeit zwischen 50ms und 200ms wechseln**.
   wartezeit = 200 → Taster drücken → wartezeit = 50 → Taster drücken → wartezeit = 200 → usw.

Zufallseffekt

CHALLENGE



- Mit der Funktion **random()** können zufällige Zahlen in einem definierten Bereich erzeugt werden.

```
// erzeugt zufällige Zahl zwischen 1 und 24
int anzahl = random(1, 25)
// erzeugt zufällige Zahl zwischen 0 und 255
int rot = random(0, 256) // Zahl zw. 0 und 255
```

Aufgabe 12:

1. Der **gesamte Ring** soll nach jedem Tastendruck eine **zufällige Farbe** darstellen.
2. **Eine (1) zufällige LED** des Rings soll nach jedem Tastendruck eine **zufällige Farbe** darstellen.
3. Eine **zufällige Anzahl** an LEDs sollen nach jedem Tastendruck eine **zufällige Farbe** darstellen.

CCA – COMPETENCE CENTRE
HTL Anichstraße

ANMELDUNG
für das
Schuljahr
2026/27

AB SOFORT
ONLINE MÖGLICH!



Wirtschaftsingenieure
Betriebsinformatik

Software entwickeln.
Technik beherrschen.
Wirtschaft durchschauen.

Möchtest du zu Hause weiterprogrammieren?

Um zu Hause weitermachen zu können, benötigst du nur einen **Windows** (empfohlen)**, Linux** oder **macOS** Rechner, sowie Zugriff auf das online Projektverzeichnis auf GitHub. Dort ist die gesamte **Installation Schritt für Schritt** beschrieben:

1. Zugriff auf das Online Repository des Projektes:
   - **https://github.com/wi-schnuppertage/smartlight**
   - Führe einfach der Reihe nach **alle Punkte** in den Kapiteln 2 bis 4 der README Datei aus.



CCA – COMPETENCE CENTRE
HTL Anichstraße

Vielen Dank, dass du da warst!

Wir freuen uns, dich im Herbst bei uns wiederzusehen!

weitere Infos